

Internal APIs Were Not All You Needed

A Signed, Layer-Descending Stack for Agent Computation:
Identity, Zero-Knowledge Credential Binding, and Bonded Route Trust

Lewis Tham
Unbrowse AI
lewis@unbrowse.ai

March 2026

Abstract

The field agreed, sensibly, that to let an agent use the web you mainly need a good search API and a tidy way to call internal endpoints. This is true. It is also, with apologies to a more famous paper,¹ *not all you needed*. An agent stops being a polite HTTP client the instant it has to click something, run a shell command, open a real browser, or put bytes on a socket that a bot-detector is staring at. The interesting work does not politely confine itself to the layer where JSON comes out. This paper makes one claim, and tries not to oversell it: the discipline that makes a *single* signed request trustworthy — sign it, seal it, cache it, prove who you are without showing your hand — is the same discipline, unchanged, all the way down the stack (screen → browser → CLI → OS → kernel → packet). Every layer is signed by one Ed25519 wallet key; credentials are bound to that identity by zero-knowledge proof and revealed only under signature; every intermediate result is content-addressed and cached, with the signed commitment appended to a hash-chained ledger (value off-chain, root on-chain); and the routes other agents lean on are secured by stake you can actually lose. We cite a real primitive for each layer instead of inventing one, and we label what is **[shipped]** versus **[proposed]** so the reader always knows whether they are looking at a product or a promise. The short version: internal APIs were a great first layer. They were never the last one.

1 Introduction: one layer is not the stack

The agent-tooling world converged on a genuinely good wedge: stop paying a browser to re-derive the web on every call, and instead look up a reusable route and just execute it [3]. Unbrowse does exactly that — intent → ranked endpoint shortlist → execute — and it wins on cost, latency, and reliability. We are not here to relitigate that. It works. **[shipped]**

We are here about the quiet assumption underneath it: that the load-bearing part of agent work lives at the HTTP layer, where things are clean. They are not clean. A login form is a screen-layer fact. A session cookie is a browser-layer fact. A keychain entry is an OS-layer fact. A TLS fingerprint is a packet-layer fact that a defender is fingerprinting whether or not you were thinking about it [31, 32]. An agent that is actually *sovereign* over its own actions has to be the same coherent entity across all of those layers — not just the one that happens to return `application/json`.

So the thesis fits on one line: *the layering is the point, and the same covenant holds at every layer*. The end-to-end argument in system design [4] already told us where each function belongs;

¹The title is an affectionate mugging of “Attention Is All You Need.” Yes, we know. That *is* the bit.

we are simply taking it at its word and running it down a signed agent stack to see if the shape survives. (It does. That is the paper.)

2 Related work

Our contribution is not a new agent or a new chain; it is the claim that one signed discipline holds across every layer an agent touches, and that the route economy is its natural settlement. We situate that against four lines of prior work.

Web agents and the browser-first assumption. A large body of work has agents drive real browsers: ReAct interleaves reasoning and action [6], and benchmarks such as Mind2Web [7], WebArena [8], and the GPT-4V grounding study SeeAct [9] measure agents on live or simulated sites. This work treats the rendered page as the interface. We take the opposite stance the Maintenance Network paper argues [3]: the browser is a fallback, and the durable interface is the first-party API the page already calls. Our stack keeps the browser as the bottom of a descent, not the default.

Tool and API invocation. Toolformer teaches a model *when* to call an API [10]; Gorilla teaches it to emit *correct* calls across thousands of APIs [11]; the Model Context Protocol standardises how a model connects to tools and data [12]. These solve single-agent tool use. They do not address what happens when many agents discover the same interface repeatedly, who maintains that knowledge as sites drift, or how the cost of discovery is shared — the maintenance-and-economics problem this paper’s later layers take up.

Identity, transparency, and capabilities. Our trust layer reuses established cryptographic systems rather than inventing them: EdDSA signatures [14], zero-knowledge credential binding [17, 16], Certificate-Transparency-style append-only logs [21], the object-capability model [29], and the Dolev–Yao adversary [13]. The novelty is not the primitives but their *composition* into one identity that signs every layer and one ledger that records every claim, with ERC-8004 [15] as the cross-agent identity surface.

Agent payments and incentives. x402 revives HTTP 402 as a real micropayment rail [2]. Prior agent-payment work largely stops at “the agent can pay.” We extend it to “the payment routes to everyone who created the value” — indexer, domain owner, platform — with the distribution and slashing analysis carried by the Maintenance Network paper [3], and we argue (Section on FDRY) that the asset securing this must be fairly distributed for the trust to hold at all. To our knowledge the combination — signed multi-layer descent, a content-addressed-plus- ledgered substrate, and a contribution-weighted three-party settlement over x402 — is not present in prior work as a single system.

3 The stack as a self-similar tree

We model the stack as a layered tree in the OSI tradition [5], but oriented around *who is acting* rather than *which bytes move*:

screen-clicks → browser → CLI → OS → kernel → packet.

The end-to-end argument [4] supplies the rule for descent: a function lives at the highest layer that can do it completely and correctly, and lower layers are fallback, not first resort. A cached signed plan beats a CLI call beats a browser action beats moving a mouse across pixels like it is 2004. But when the higher layer simply cannot act — no API, hostile anti-bot, a page that is 100% JavaScript and 0% mercy — the agent descends, and at the bottom it emits packets whose TLS/HTTP fingerprint is indistinguishable from a real browser via curl-impersonate [30], matching exactly the JA3/JA4 signature the other side is keying on [31, 32]. [shipped] for browser/CLI/HTTP and fingerprint-faithful fetch; [proposed] for a uniform *signed* descent through the OS and kernel layers.

The reason any of this composes is that the shape at every layer is identical: observe, decide, sign, act. That self-similarity is the whole trick — it is what lets *one* identity and *one* cache serve the entire tree instead of six bespoke integrations held together with hope and retry loops.

This is not a metaphor we are stretching. The whole stack is one instance of a single structure — the covenant tree from which our internal tooling is built — and every section below is one atom of it, mapped to a primitive that already exists in the literature (Table 1). We did not invent these; we aligned them.

layer / atom	primitive	source
descent (walk)	curl-impersonate / JA3, JA4	[30, 31, 32]
network (economics)	x402 split settlement / fair three-way compensation	[2, 3]
agent record (node)	ERC-8004 trustless agent	[15]
the stack (tree)	end-to-end argument / OSI	[4, 5]
witness (no leak)	zkLogin / Pedersen commit	[17, 19]
cache (value)	Merkle content-addressing	[20]
↔ KV parallel	attention KV cache / PagedAttention	[25, 26]
seal (no unsigned)	AEAD / object-capabilities	[28, 29]
ledger (commitment)	Certificate Transparency / hash chain	[21, 22]
settle (bonded)	bonded route trust / FROST quorum	[3, 33]
loop (control)	OODA / proof-of-history clock	[34, 23]
descent (walk)	curl-impersonate / JA3, JA4	[30, 31, 32]

Table 1: Each layer of the signed stack is one atom of the covenant tree, mapped to an existing, cited primitive. A runnable reference implementation of the cache + ledger core ships alongside this paper (reference/, with tests that execute each claim).

4 Identity: one key signs every layer

The root of trust is a single Ed25519 keypair [14] — and conveniently, the agent already has one, because a Solana account literally *is* a base58 Ed25519 public key. The wallet is the identity. Every action — a click, a syscall, a packet — is admissible only if it chains to a signature under that one root. No signature, no action; the stack has no anonymous side door. [shipped]: the Unbrowse signer already produces wallet signatures for resolve/execute admission.

For trust *between* agents we adopt the ERC-8004 “Trustless Agents” model [15], which defines three on-chain registries: **Identity** (portable agent IDs), **Reputation** (signed feedback), and **Validation** (independent re-execution / ZK / TEE checks). An Unbrowse route maintainer is precisely one of these agents: a portable identity that can carry reputation and be checked by someone who does not trust it yet. [proposed]: the ERC-8004 binding is a design target, not a shipped feature, and we will not pretend otherwise three paragraphs from a contract address.

5 Witness without disclosure: ZK credential binding

Signing is the easy half. The hard half — the actual research — is proving a credential belongs to the identity *without revealing the credential*. Your password, your session cookie, your 1Password entry, the fact that you clicked “approve” for this domain: all of it should be provably bound to the wallet and leak *nothing* identifiable at any layer unless you choose to open the proof under signature.

This is the classical anonymous-credential problem [16], and it has grown up. zkLogin [17] binds an existing OAuth/OpenID identity to a chain address through a Groth16 proof, hiding the link even from the identity provider; Semaphore [18] proves anonymous group membership with zk-SNARKs; Pedersen commitments [19] give the “commit now, open later, can’t lie about it” primitive for a single secret. Together they make the binding a two-witness corroboration that betrays nothing: the chain sees *that* a credential is bound, never *what* it is. [proposed] — this is the central new proposal of the paper, and to be blunt, today Unbrowse stores credentials in a boring local vault like everyone else. The proposal is the upgrade, not the status quo.

6 Cache: content-addressed, sealed unless revealed

Every layer’s settled result — a resolved endpoint, a rendered page, a signed plan — is memoised under a content-addressed key, in the Merkle tradition [20] and exactly as realised by content-addressed stores like IPFS [24] and Git’s object model [27]. Same structure re-walked, same key, same answer, deterministically; the cache is re-derived to *verify*, never trusted because it looked right last time. Unbrowse’s centralised cache servers can hold these entries and serve them across agents, while a Pedersen commitment [19] over a sensitive entry keeps it sealed until the holder opens it — a cache that is public by default and private by proof. [shipped] for endpoint/route caching; [proposed] for the sealed-unless-revealed commitment layer.

The parallel worth naming is to the *KV cache* inside the transformer doing the reasoning. Attention [25] computes a key and value for every token; the KV cache memoises those states so decoding never recomputes them, and modern serving engines page that cache like OS virtual memory and *share identical prefixes across requests* by content (hash) key [26]. That is the same discipline, one altitude up: same prefix, same cached blocks, reused not recomputed — except the structural key is now a covenant subtree rather than a token prefix, and the entry is ZK-sealed rather than served in the clear. The model caches computation it has already done; the stack caches *actions* it has already signed. Same shape, different altitude — a KV cache all the way down, where each layer’s entry stays sealed until a signature opens it.

7 Seal: nothing ships unsigned

Before any layer emits, it self-verifies through the root. At the wire that is authenticated encryption with associated data (AEAD) [28]: the receiver checks the tag before it trusts a single byte, and tampering fails closed rather than failing quietly into your logs. The same discipline generalises upward as object-capability attenuation [29] — authority travels only by reference and can be narrowed, never picked up from the ambient air. The rule is boring on purpose: no unsigned action crosses the gate, at any layer, ever. [proposed] as a *uniform* stack property — AEAD itself is, obviously, ubiquitous and shipped at the transport layer; we are not claiming to have invented encryption.

8 Ledger: where the signatures actually land

Signing is only half the story; the obvious next question — and the one most “signed” systems wave away — is *where does the signature go*. A cache and a ledger are not the same object, and conflating them is the bug. A cache is content-addressed: you fetch a value by the hash of its content and order does not matter, so the same hash resolves on any host [20]. A ledger is the orthogonal half: append-only, *ordered*, and hash-chained, so the history and the order are themselves tamper-evident. A signed entry — a signed KV result, a route attestation, a bonded quality claim — needs both: the cache holds the value, the ledger holds the signed commitment to it.

The load-bearing design choice is *value off-chain, root on-chain*. You never put the payload on a chain — that would be paying gas to store megabytes and reading them slowly. You store the small, signed commitment: an entry of {**signer**, **value-hash**, **timestamp**, **prev-hash**}, where the value itself lives in the content-addressed cache and only its hash appears in the log. Each entry chains to the prior, so reordering or editing any past row breaks every root after it [22]. Independent auditors — not a trusted operator — verify the log is append-only and consistent, exactly the model Certificate Transparency formalises with its Merkle log, Signed Tree Head, and inclusion proofs [21]. That is what makes the ledger trustless rather than “trust our server.”

This also answers how the system decentralises without a rewrite. Per-entry on-chain writes do not scale, so you batch: thousands of entries collapse to one Merkle root, and only the *root* is checkpointed on-chain (the Certificate-Transparency / rollup pattern), ordered by a decentralised clock such as Solana’s proof of history [23]. The maturity ladder is one shape at every rung: a local append-only log today, a signed-root server next, on-chain Merkle-root checkpoints as the decentralised end-state — you swap the host, the signatures and the hash-chain never change. [\[shipped\]](#) for the local append-only ledger (the route graph already keeps signed JSONL records); [\[proposed\]](#) for the on-chain root checkpointing and independent-auditor layer.

9 Settle: bonded, slashable route trust

A route graph that carries real traffic has a maintenance problem, not just a discovery problem: routes go stale, schemas drift, auth flows mutate the week after you cache them [3]. Discovery itself is settled by the inequality

$$f_{\text{shared route}} < c_{\text{rediscovery}},$$

under which a rational agent prefers the shared route whenever its fee undercuts the expected cost of rediscovering it [3]. Maintenance needs something with teeth: *collateralised accountability* — a maintainer signs a claim that a route is fresh and safe, and a false or conflicting claim is slashable. The mechanism and its incentive analysis are the subject of the Maintenance Network paper [3]; threshold signatures (FROST [33]) finalise a trust-tier claim only with a *t*-of-*n* quorum rather than one confident wallet. In Unbrowse terms this is the FDRY layer [3]: stable settlement for ordinary usage, with a native asset reserved *strictly* for the security function — bonding accountable maintenance, funding challenges, slashing liars. Bonding buys credibility and eligibility; it explicitly does *not* buy ranking, because the moment stake buys placement you have built a pay-to-win search engine and deserve the users you get. [\[proposed\]](#) — FDRY’s bonding/challenge layer is sequenced as future work in [3], and nothing in the wedge depends on rushing it.

The token contract address, since a whitepaper that hides its own CA is just a PDF with anxiety, is 2ZiSPGncrkwWa6GBZB4EDtsfq7HEWwkwsPFzEXieXjNL on Solana.²

²We put the contract address in the paper on purpose. If you would rather we were coy about it, there are many other whitepapers.

10 The network: who pays, who earns, who opts in

A signed, ledgered route graph is infrastructure, and infrastructure has an economy. We are explicit about it because a trust paper that is coy about money is not actually trustworthy — but we describe the *what*, not the engine that makes capture cheap, which is the part we keep.

Commoditised RPA, routed through one layer. Every site that an agent automates today is re-discovered from scratch, per agent, per call — the browser tax. Unbrowse turns that one-off robotic-process-automation work into a *shared, maintained route graph*: discover once, reuse forever, and route the API/search cost through one infrastructure layer rather than re-paying it everywhere. The graph, kept fresh by real traffic, is the asset; any single route is copyable, the maintained graph is not [3]. [shipped] for resolve/execute over the shared graph; the maintenance-network economics are sequenced in [3].

Execution is the payment surface for security. Payment is bound to the act, not bolted on: a paid `execute` settles over `x402` [2] — the HTTP 402 “Payment Required” protocol revived as a real rail — through a split authorisation, and that settlement is what funds the substrate that keeps the graph safe to depend on (validation, freshness, sponsor admission). `x402` is the right primitive precisely because it is the missing payment layer the web stubbed out in 1997: a call either carries a valid micropayment or it gets a 402, so charging per execution needs no accounts, invoices, or middleman — the wire itself settles. The agent either pays per call on the wallet rail or rides a subscription that hides the chain entirely; either way the platform share funds the security layer, not a rent-seeker. [shipped] (the `x402` split settlement and subscription admission are shipped).

Websites get wallets they opt into. The quiet structural move: every paid call on a route splits three ways — the indexer who captured it, the *owner of the domain it serves*, and the platform. A domain’s share accrues the moment its routes earn, *before the owner has done anything*, held in a public, per-domain-attributed escrow. To claim it, the owner proves the domain is theirs the same way they would opt out — a DNS record or a `.well-known` file — and binds it to a wallet via Privy domain sign-in. From the next settlement on, their share routes straight to them. The incentive runs the right way: the network pays domains for being automated well, and the only way to collect is to join the network. [proposed]/[shipped] — the fair-split, public earnings ledger, and Privy-DNS claim flow are documented and partially shipped; the full website-wallet rollout is sequenced, not claimed as universal today.

The game theory: pay whoever actually created the value. The split is not charity, it is incentive design. Three parties produce the value of a reused route, so `x402` routes USDC to all three in one settlement: the *indexer* who did the work of discovering and capturing the route, the *domain owner* whose service is being called (paid for hosting the thing worth automating), and the *platform* that runs the routing, search, and security substrate. Each share is the dominant strategy for the role it pays: an indexer earns by capturing routes others reuse, so the graph’s coverage grows; a domain owner earns by being reliably automatable, so blocking agents becomes the costly move and cooperation the profitable one; the platform earns only when a settlement it secured actually happens, so its incentive is the graph staying trustworthy, not extracting rent. Because the shares derive from on-ledger contribution metrics rather than a hardcoded pricing table, no party can game a per-request knob — the hardest-to-fake signal, realised payment, is what ranks and rewards [2]. The same `x402` rail can carry staking rewards: USDC accrued by a staked role flows

back through the identical split mechanism, so “who deserves the money” is answered mechanically by who contributed, not by who asks. [\[proposed\]](#) for the staking-reward routing; the three-way live split is shipped.

Why FDRY, and why fair-launched. Usage does not need a token — it settles in stable USDC over x402 [\[2\]](#), and we are emphatic that it stays that way. FDRY exists for one job the stablecoin cannot do: the security/accountability layer, where a maintainer’s bond, a challenger’s stake, and a slashing penalty must be denominated in an asset whose value is tied to the network’s own integrity [\[3\]](#). The non-obvious constraint is on its *distribution*, and it falls straight out of the witness atom: accountability is credible only when many *independent* parties hold the bond, the same way a claim stands on two independent witnesses rather than one. A security asset concentrated in insider hands is a single witness wearing many hats — it contradicts the very decentralisation it is meant to underwrite. So FDRY launched fair by necessity, not branding: no pre-sale, no team allocation, no insider round; supply placed on a public Solana pool at launch, acquirable only on the open market at the same price for the founder as for everyone else. The platform’s USDC fee is the sap that flows back to whoever holds that distributed security stake — staking by abiding, not by lock-up. [\[shipped\]](#) (fair launch + live platform-fee collection); [\[proposed\]](#) (the vault cycle that returns the fee to holders). No promise of profit; the mechanism is the claim, not a guarantee.

Why money could not be the point. There is a deeper reason the distribution had to be fair, and it is not financial — it is about the order in which trust and money can be created. A network that puts money first inverts its own foundation: if the token is launched to enrich its issuer, then the issuer’s incentive at the root is extraction, and every participant downstream correctly discounts the system’s trust claims accordingly — the rot is at the origin, so no amount of mechanism above it recovers credibility. This is the same failure societies hit when an institution’s first move is to capture rent: the capture is observed, and trust never accrues. The accountability layer we need — proof-of-stake bonding and ZK identity — *cannot* sit on a money-first root, because its entire job is to let strangers trust each other’s claims, and a root optimised for the issuer’s gain is exactly the thing strangers are right to distrust. FDRY clears this only because money was not its point: it was distributed with no issuer advantage and then *repurposed* as the security stake. The asset earns its trust the same way a route does in this system — by what it does for the network, corroborated by independent holders, not by what it promised its founder. Trust precedes value here; the value is the fruit of the trust, never its bait.

What stays ours. Everything above is the *what*: the graph, the split, the claim. The *how* — the capture, indexing, and platform-integrity primitives that make route discovery cheap and the graph fresh — is closed by design [\[1\]](#), for the abuse reasons stated there. The boundary is not secrecy for its own sake; it is the difference between giving agents real APIs and handing out an unattributed scraping fleet. This paper, and the security audit it supports, surface the trust surface in full and stop exactly at that engine.

Reproduced mechanism (this paper’s own measurement). The field result above is the live-web study; we additionally ship a runnable micro-benchmark (`reference/bench/bench_reuse.py`) that isolates and measures the *mechanism* the field speedup rests on — content-addressed reuse versus re-derivation — on the actual cache implementation. Over 60 trials, re-deriving a representative ~64 KB extracted payload averages **4.48 ms** while a content-addressed cache hit (read

+ hash re-verify) averages **0.052 ms**: a measured $\approx 95\times$ mean speedup, wall-clock, deterministic, with a gate that fails if reuse is not materially faster. We are careful about what this is: it is a CPU-only recompute-versus-reuse demonstration and is therefore *larger* than the $3.6\times$ field figure, which includes the browser, the network, and real page variance the micro-benchmark deliberately excludes. The field number is the honest end-to-end claim; this number proves the structural reason behind it — reuse is asymptotically cheaper than rediscovery — holds on our code, not just in prose. primitives that make route discovery cheap and the graph fresh — is closed by design [1], for the abuse reasons stated there. The boundary is not secrecy for its own sake; it is the difference between giving agents real APIs and handing out an unattributed scraping fleet. This paper, and the security audit it supports, surface the trust surface in full and stop exactly at that engine.

11 The control loop

The whole stack runs as an OODA loop [34]: observe the layer’s state, orient it against the model (this is where the learning actually lives, despite “decide” getting all the press), decide the next action at the right layer, act under signature — then repeat with feedback, dropping a layer or replanning on failure, and resting when the goal settles. It is the same plan/build/test/judge cycle, run uniformly at every layer of the tree. If that sounds unglamorous, good: the glamorous control loops are the ones that do not converge.

12 Evaluation

We summarise the quantitative result the wedge rests on, as reported in the Maintenance Network study [3], and are precise about what it does and does not establish.

Setup. A single-host, live-web benchmark over **94 domains** compares two ways of completing the *same* information-retrieval task: (i) executing a cached first-party route, versus (ii) driving a real browser (Playwright) to obtain the equivalent result. Tasks are matched for output so the comparison is like-for-like, not API-easy-case versus browser-hard-case.

Results. Cached route execution averages **950 ms** against **3,404 ms** for browser automation — a **$3.6\times$ mean** speedup, with a **$5.4\times$ median** (the median exceeds the mean because the browser’s tail is heavy: slow pages drag its average up, while cached routes are tightly distributed). Well-cached routes complete in under **100 ms**. The local cache+ledger reference implementation that ships with this paper is separately unit-verified — content-addressing, value-off-chain/root-on-chain, hash-chain tamper-evidence, ed25519 signatures, and deterministic Merkle-root inclusion all pass as executable tests, so the substrate’s correctness claims are runnable, not asserted.

Live single-URL spot-check (this session). As an honest sanity check — not the field study — we timed the shipped `unbrowse v7.0.2` binary on one URL in a sandbox: a cold discovery (**resolve**, cache miss $\rightarrow$ marketplace/search) reported **5396 ms**, while a warm cached fetch (HTTP replay of a known route) reported **201 ms** — a $\approx 27\times$ observed gap, both numbers read from the binary’s own output (`reference/bench/live-measured-1url.json`). This is $n=1$ and compares discover-vs-replay rather than browser-vs-execute; it is directionally consistent with the field result (cached $\gg$ cold) but is explicitly *not* a reproduction of the 94-domain $3.6\times$ headline, which remains future work.

What this establishes, and what it does not. The measurement supports exactly one claim: when a route already exists in the graph, reusing it is substantially cheaper than rediscovering it with a browser — which is the entire economic premise, since an agent adopts the shared graph only when the route fee undercuts expected rediscovery cost [3]. It does *not* establish cold-start discovery cost (the first agent still pays the browser tax), end-to-end task success rates (not reported), or accuracy on adversarial or heavily-JavaScript-gated sites where the descent to the browser layer is forced. We state this plainly: the speedup is a reuse result, not a universal one, and the honest adoption condition is the inequality of the network section, not a blanket “always faster.”

13 Threat model

We state the adversary explicitly, because a trust paper that never names what it defends against is decoration. We assume a Dolev–Yao network attacker [13] — able to read, drop, replay, and forge messages on any layer — plus three application-level adversaries the route economy specifically invites. For each, we name the atom that resists it and the residual risk we do not claim to close.

(A1) The impersonator. An attacker replays or forges an action to act as another identity. Resisted at the *root*: every action carries an Ed25519 signature over its canonical bytes [14], and the ledger’s `prev-hash` makes a replayed entry detectable out of order [21]. Residual: key theft is out of scope — a stolen wallet key is the user, by construction. This is the same boundary every signing system accepts.

(A2) The credential thief / over-reach. An operator retargets imported sessions at accounts they do not own — the abuse vector that closed the source [1]. The *witness* atom narrows it: credentials are bound to the identity by zero-knowledge proof and revealed only under signature [17, 16], so a leaked ledger or cache exposes *that* a credential is bound, never the credential. Capability attenuation [29] bounds what a delegated action may do. Residual: an operator authenticated as themselves can still drive their own credentials; we constrain reach, not self-harm.

(A3) The poisoner. An adversary publishes a false or stale route to mislead later agents — the integrity attack on a shared graph. Resisted at *settle* and *witness* together: a trust-tier route carries a slashable bond and is challengeable on independent evidence [3], and finalisation needs a *t*-of-*n* quorum [33], so one dishonest maintainer cannot settle a claim. Residual: open (unbonded) routes remain best-effort by design; the model moves trust-sensitive traffic to bonded routes, it does not claim the open tier is adversary-proof.

(A4) The free-rider and the Sybil. An actor consumes the graph without paying, or splits into many identities to farm rewards. The *verb* atom binds payment to the act: an unpaid `execute` gets HTTP 402, not service [2]. Sybil farming is bounded because reward shares derive from on-ledger contribution metrics, not headcount — minting identities does not mint contribution. Residual: collusion between an indexer and a domain owner to inflate a route’s apparent value is a real open problem; we flag it rather than wave it away, and it is the strongest argument for the bonded tier’s slashing teeth.

(A5) The exfiltrator of the moat. An adversary — or an honest contributor by accident — leaks the closed capture/integrity engine into a public artifact. Resisted mechanically, not by policy: `leak-guard.sh` and `paper-gate.sh` run in release CI and fail the build if a sensitive term

or an unanchored claim reaches a public path. This paper is itself subject to that gate. Residual: NDA source review remains the only full-disclosure path, by the abuse reasoning of [1].

What we do not defend. We make no claim against a malicious endpoint that lies in its own responses (a route can faithfully replay a hostile API), a global passive adversary correlating timing across the whole network, or coercion of a key holder. These are out of scope and named so the reader is not misled by silence.

14 What is actually built (no fabricated green)

In the spirit of not selling a roadmap as a changelog:

- [\[shipped\]](#) Intent → route resolve → execute; live browser capture; HTTP fetch with browser-faithful TLS fingerprinting; wallet-signed admission; route/endpoint caching.
- [\[proposed\]](#) Uniform *signed* descent through the OS/kernel layers; ZK credential binding (the central contribution); the sealed-unless-revealed commitment cache; ERC-8004 registry binding; FDRY bonding / challenge / slashing.

Treat the [\[proposed\]](#) list as a research agenda with cited foundations, not a feature list with a release date. The gap between those two things is the entire difference between a whitepaper and a press release, and we would like to stay on the correct side of it.

15 Conclusion

Internal APIs were an excellent first layer — enough to prove the wedge in a real market, which is more than most ideas manage. They were never going to be the last layer, because an agent that is sovereign over its own actions has to be one coherent entity across every layer it touches: under one key, revealing nothing it did not choose to reveal, and standing behind its routes with something it can actually lose. The contribution is deliberately modest — one covenant (sign it, seal it, cache it, bind identity without disclosure) that holds all the way down, each layer pinned to a real primitive. Attention, it turns out, was a great deal of what you needed. It just was never the *stack*.

References

- [1] Unbrowse AI. *Open Source Notice — the closed/open boundary*. docs/OPEN-SOURCE-NOTICE.md, 2026.
- [2] Coinbase. *x402: An Open Protocol for Internet-Native Payments over HTTP 402*. x402.org whitepaper, 2025. <https://github.com/coinbase/x402>.
- [3] L. Tham. *Unbrowse Maintenance Network: Trust, Accountability, and Optional Bonding in a Shared Route Graph*. arXiv:2604.00694, 2026.
- [4] J. H. Saltzer, D. P. Reed, D. D. Clark. *End-to-End Arguments in System Design*. ACM TOCS 2(4):277–288, 1984. DOI: 10.1145/357401.357402.
- [5] ITU-T Recommendation X.200 (= ISO/IEC 7498-1), *OSI Basic Reference Model*, 1994.

- [6] S. Yao, J. Zhao, et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. arXiv:2210.03629.
- [7] X. Deng, Y. Gu, et al. *Mind2Web: Towards a Generalist Agent for the Web*. NeurIPS 2023. arXiv:2306.06070.
- [8] S. Zhou, F. F. Xu, et al. *WebArena: A Realistic Web Environment for Building Autonomous Agents*. ICLR 2024. arXiv:2307.13854.
- [9] B. Zheng, B. Gou, et al. *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*. ICML 2024. arXiv:2401.01614.
- [10] T. Schick, J. Dwivedi-Yu, et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS 2023. arXiv:2302.04761.
- [11] S. G. Patil, T. Zhang, et al. *Gorilla: Large Language Model Connected with Massive APIs*. NeurIPS 2024. arXiv:2305.15334.
- [12] Anthropic. *Model Context Protocol*. 2024. <https://modelcontextprotocol.io>.
- [13] D. Dolev, A. C. Yao. *On the Security of Public Key Protocols*. IEEE Trans. Information Theory 29(2), 1983. DOI: 10.1109/TIT.1983.1056650.
- [14] S. Josefsson, I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, IRTF CFRG, 2017.
- [15] *ERC-8004: Trustless Agents*. Ethereum Improvement Proposals (Draft). <https://eips.ethereum.org/EIPS/eip-8004>
- [16] J. Camenisch, A. Lysyanskaya. *An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation*. EUROCRYPT 2001, LNCS 2045. IACR ePrint 2001/019.
- [17] F. Baldimtsi, K. Chalkias, et al. *zkLogin: Privacy-Preserving Blockchain Authentication with Existing Credentials*. ACM CCS 2024. arXiv:2401.11735.
- [18] Semaphore Protocol. *Semaphore: anonymous signaling on Ethereum*. <https://github.com/semaphore-protocol/semaphore>
- [19] T. P. Pedersen. *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*. CRYPTO 1991, LNCS 576. DOI: 10.1007/3-540-46766-1_9.
- [20] R. C. Merkle. *A Digital Signature Based on a Conventional Encryption Function*. CRYPTO 1987, LNCS 293. DOI: 10.1007/3-540-48184-2_32.
- [21] B. Laurie, A. Langley, E. Kasper. *Certificate Transparency*. RFC 6962, IETF, 2013.
- [22] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. <https://bitcoin.org/bitcoin.pdf>
- [23] A. Yakovenko. *Solana: A New Architecture for a High Performance Blockchain*. 2018. <https://solana.com/solana-whitepaper.pdf>
- [24] Protocol Labs. *IPFS / IPLD Merkle-DAG specifications*.

- [25] A. Vaswani, N. Shazeer, et al. *Attention Is All You Need*. NeurIPS 2017. arXiv:1706.03762.
- [26] W. Kwon, Z. Li, et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. arXiv:2309.06180; <https://github.com/vllm-project/vllm>.
<https://github.com/ipfs/specs>
- [27] S. Chacon, B. Straub. *Pro Git*, 2nd ed., §10.2 Git Internals — Git Objects. <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>
- [28] D. McGrew. *An Interface and Algorithms for Authenticated Encryption*. RFC 5116, 2008.
- [29] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.
- [30] curl-impersonate. *A special build of curl that impersonates real browsers (TLS/HTTP fingerprints)*. <https://github.com/lwthiker/curl-impersonate>
- [31] Salesforce. *JA3: TLS client fingerprinting*. <https://github.com/salesforce/ja3>
- [32] FoxIO. *JA4+ network fingerprinting suite*. <https://github.com/FoxIO-LLC/ja4>
- [33] C. Komlo, I. Goldberg. *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. SAC 2020, LNCS 12804. IACR ePrint 2020/852; RFC 9591.
- [34] J. R. Boyd. *The Essence of Winning and Losing*, 1995; in *A Discourse on Winning and Losing*, ed. G. Hammond, Air University Press, 2018.